Credit Card Fraud Detection

1. Project Introduction

Credit card fraud is a major problem in digital financial transactions. With a large number of transactions taking place every day, manually identifying fraudulent transactions is difficult.

The objective of this project is to use **Machine Learning classification algorithms** to identify whether a credit card transaction is **Fraudulent or Non-Fraudulent**.

The project follows an end-to-end Machine Learning process:

Data Understanding → Data Cleaning → EDA → Feature Engineering → Class Imbalance Handling → Logistic Regression → Algorithm Comparison → Model Evaluation → Best Model Selection

Project Objectives

- Understand the credit card transaction dataset.
- Clean and preprocess the data.
- Perform Exploratory Data Analysis.
- Create useful features.
- Check Fraud vs Non-Fraud transactions.
- Handle class imbalance using Under Sampling and Over Sampling.
- Check whether Logistic Regression works for fraud detection.
- Compare Decision Tree, Random Forest, and KNN.
- Evaluate models using Accuracy, Precision, Recall, and F1 Score.
- Select the best-performing model.

---

2. Dataset Description

The project uses a **Credit Card Fraud Detection dataset (`test.csv`)**.

The dataset contains transaction-related information such as:

Feature	Description

`trans_date_trans_time`	Transaction date and time
`cc_num`	Credit card number
`merchant`	Merchant name
`category`	Transaction category
`amt`	Transaction amount
`gender`	Customer gender
`city`	Customer city
`state`	Customer state
`lat`	Customer latitude
`long`	Customer longitude
`city_pop`	City population
`job`	Customer occupation
`dob`	Customer date of birth
`merch_lat`	Merchant latitude
`merch_long`	Merchant longitude
`is_fraud`	Target variable

Target Variable

is_fraud = 0 → Non-Fraud
is_fraud = 1 → Fraud

---

3. Data Understanding and Cleaning

The dataset was loaded using Pandas and basic data checks were performed.

Checks performed

- Number of rows and columns
- Data types
- Missing values
- Duplicate records
- Target variable distribution
- Date format

---

4. Exploratory Data Analysis (EDA)

EDA was performed to understand transaction patterns and identify characteristics related to fraudulent transactions.

4.1 Transaction Analysis

The following were calculated:

- Total Transactions
- Total Fraud Transactions
- Fraud Percentage
- Non-Fraud Transactions

```
==================================
       TRANSACTION ANALYSIS
==================================
Total Transactions: 555719
Total Fraud Transactions: 2145
Fraud Percentage: 0.39 %
==================================
```

---

5. Amount Analysis

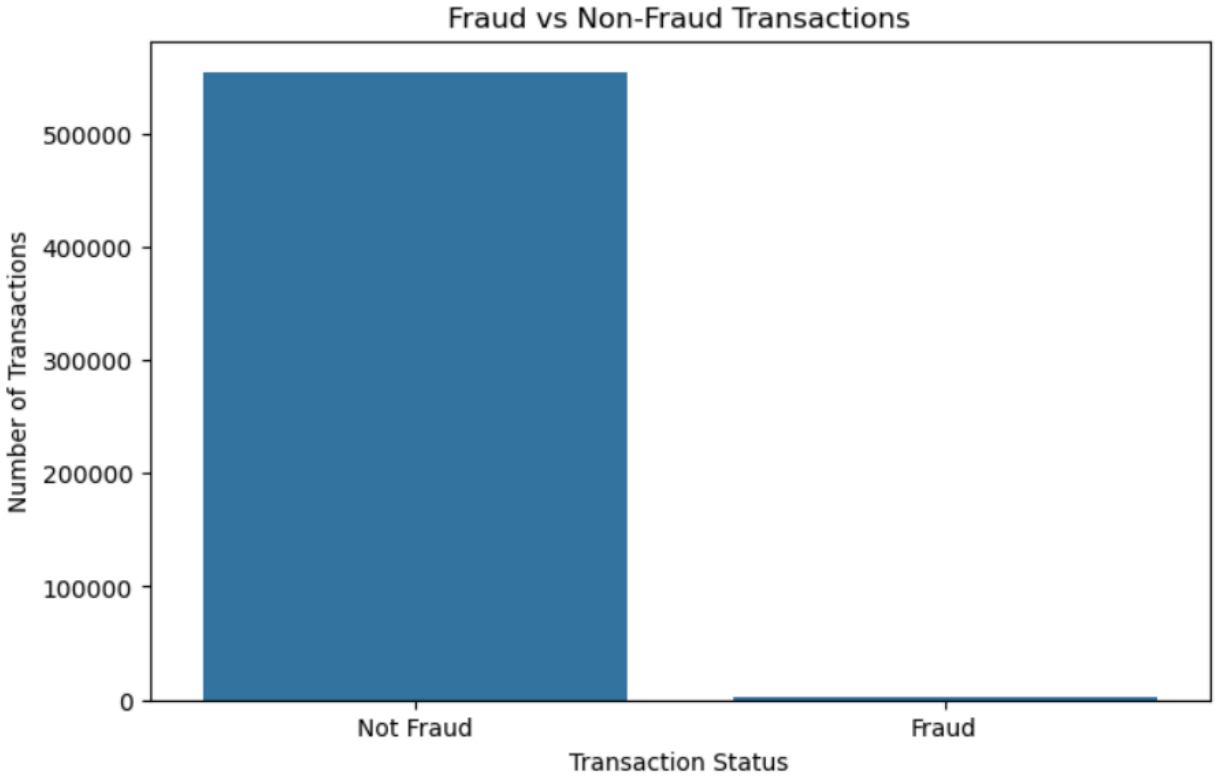
Amount Analysis helps understand the transaction value pattern in the dataset.

- **Maximum Transaction:** Highest transaction amount recorded.
- **Minimum Transaction:** Lowest transaction amount recorded.
- **Average Transaction:** Overall typical transaction value.
- **Median Transaction:** Middle transaction value, less affected by extreme values.

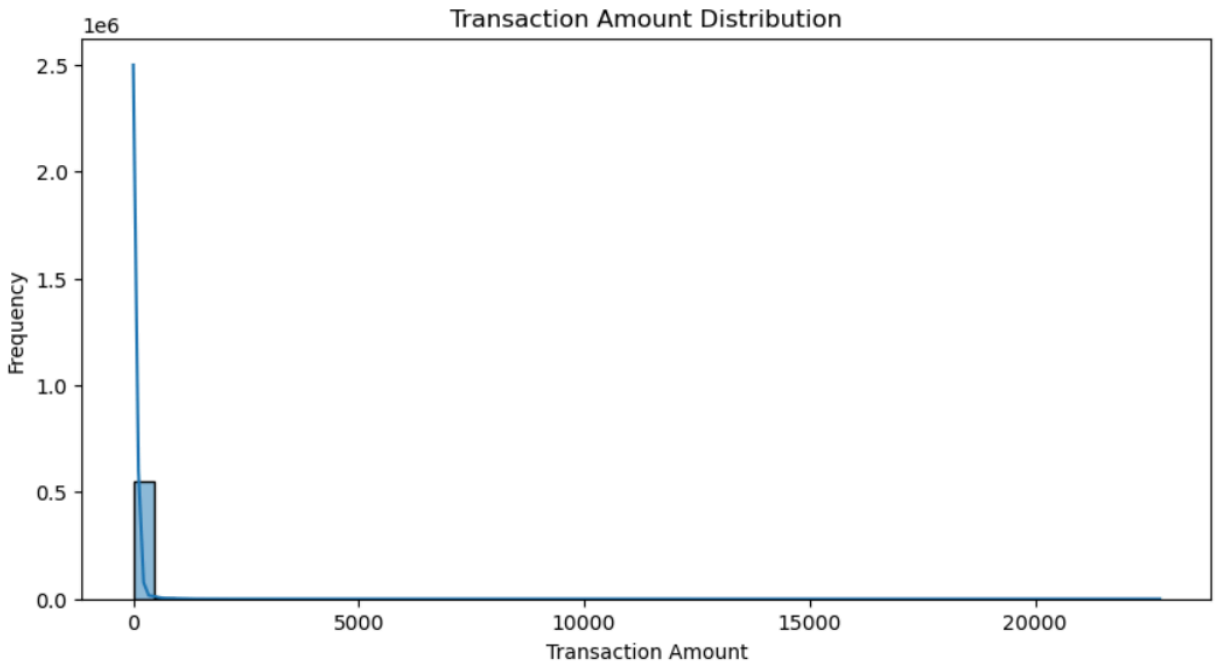
```
==================================
          AMOUNT ANALYSIS
==================================
Maximum Transaction: 22768.11
Minimum Transaction: 1.0
Average Transaction: 69.39
Median Transaction: 47.29
----------------------------------
```

Visualizations

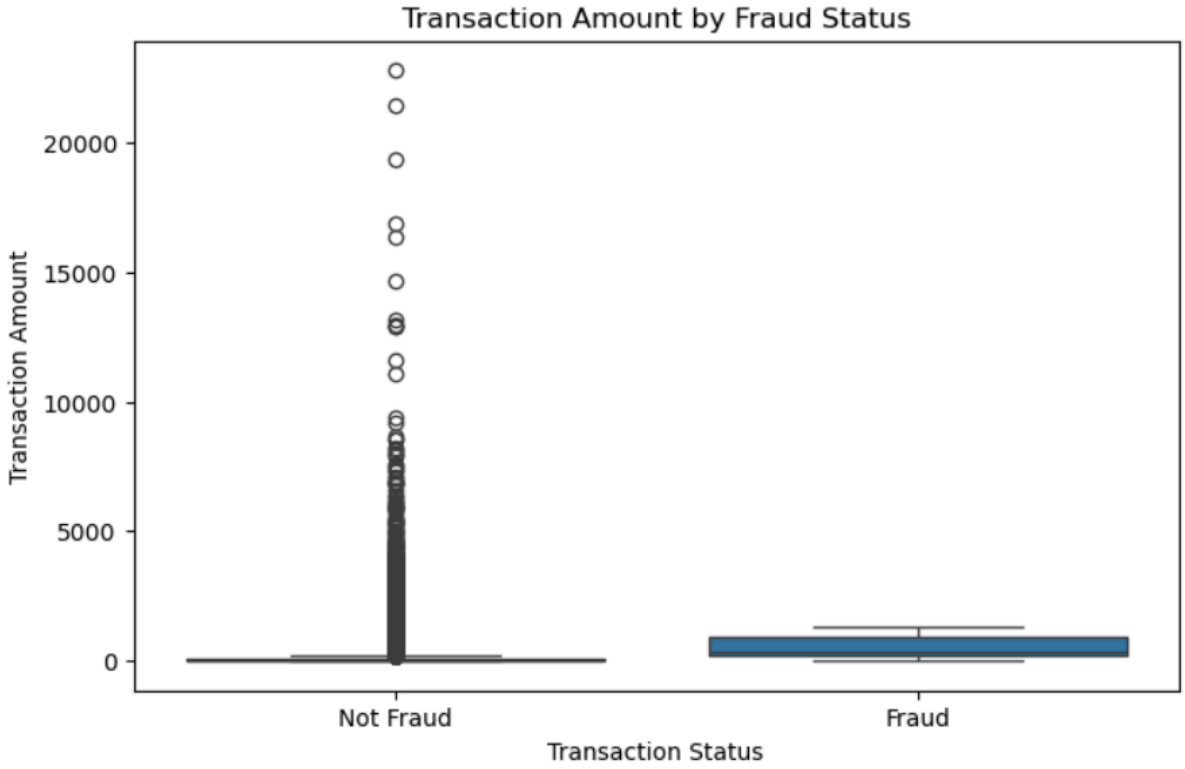
- **Fraud vs Non-Fraud**



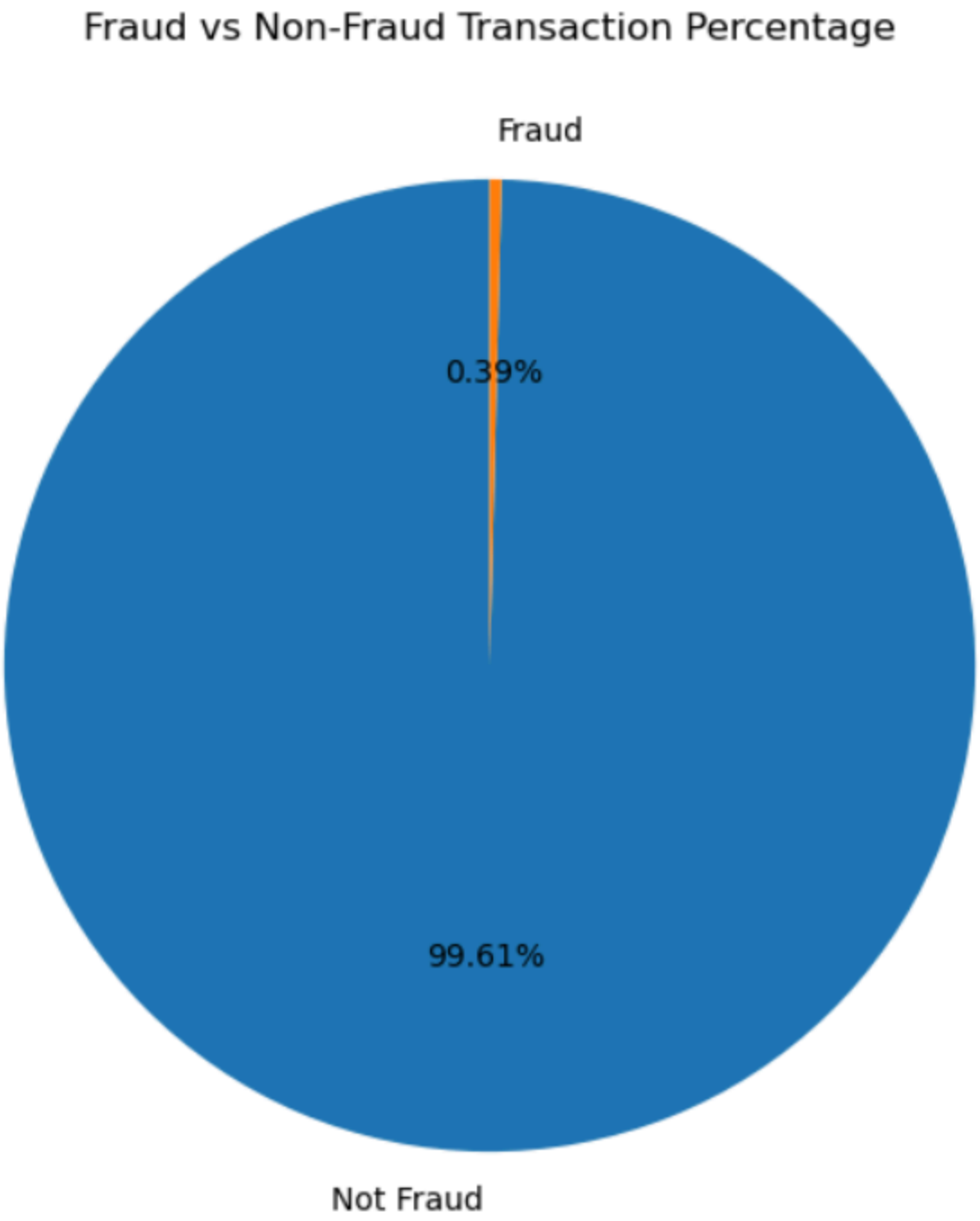
- **Histogram** → Transaction amount distribution



- **Box Plot** → Potential outliers



- **Pie Chart** → Fraud percentage



---

6. Time Analysis

Transaction time can be an important factor in fraud detection.

- **Fraud by Time Distribution:** Shows how fraudulent transactions are distributed across different hours of the day.
- **Peak Fraud Hour:** Identifies the hour when the **highest number of fraud transactions** occurs.

Time analysis helps identify **high-risk hours and unusual transaction patterns**, which can support effective fraud detection.

---

7. Feature Engineering

Feature Engineering means creating new useful features from existing data to improve fraud detection.

- **Transaction Day**: Identifies the day of the month when the transaction occurred.
- **Transaction Month**: Identifies the month of the transaction.
- **Day of Week**: Shows whether the transaction occurred on Monday, Tuesday, etc.
- **Weekend Indicator**: Identifies whether the transaction occurred on a weekend.
- **Customer Age**: Calculates the customer's age at the time of the transaction.

These features help the model understand transaction timing, weekend patterns, and customer characteristics, improving its ability to identify potential fraud.

---

8. Class Imbalance Analysis

Fraud vs Non-Fraud Count

The **Fraud vs Non-Fraud count** shows the distribution of transactions between the two classes.

- **Non-Fraud:** Majority of transactions
- **Fraud:** Minority of transactions

The dataset contains a **large number of Non-Fraud transactions and a much smaller number of Fraud transactions**.

Therefore, the dataset is called an **imbalanced dataset**.

Why is this a problem?

Suppose:

Non-Fraud → 99%
Fraud → 1%

A model that predicts every transaction as Non-Fraud could achieve approximately 99% accuracy while detecting **zero fraud transactions**.

Therefore:

> **Accuracy alone is not sufficient for fraud detection.**

We also need to consider:

- Precision
- Recall
- F1 Score

---

9. Random Undersampling

Random Undersampling balances the dataset by reducing the number of **Non-Fraud (majority class)** transactions to match the Fraud transactions.

Advantages

- Creates a more balanced dataset.
- Reduces training time and dataset size.

Limitation

- Some genuine Non-Fraud transaction information is removed.

  Undersampling helps the model focus more equally on Fraud and Non-Fraud transactions, but removing genuine data may result in loss of useful information.

---

10. Random Oversampling

Random Oversampling balances the dataset by increasing the number of **Fraud (minority class)** transactions through duplication.

Advantages

- Keeps all original Non-Fraud transactions.
- Gives Fraud transactions more representation during training.
- Helps the model learn minority-class patterns better.

Limitation

- Duplicate Fraud records can increase the risk of **overfitting**.

  Oversampling improves the representation of Fraud transactions without removing genuine data, making it useful for handling class imbalance.

---

11. Logistic Regression

Logistic Regression was first tested on the **original imbalanced dataset** to establish a baseline model.

Model Performance

Metric	Non-Fraud	Fraud
Precision	1.00	**0.05**
Recall	0.95	**0.73**
F1 Score	0.97	**0.10**

Overall Accuracy: 94.76%

Confusion Matrix

	Predicted Non-Fraud	Predicted Fraud
Actual Non-Fraud	105,001	5,714
Actual Fraud	114	**315**

Interpretation

- The model correctly detected **315 out of 429 actual fraud transactions**.
- Fraud **Recall = 73%**, which means the model detected a good portion of actual fraud cases.
- However, Fraud **Precision = only 5%**, meaning many transactions predicted as fraud were actually Non-Fraud.
- The **94.76% accuracy** looks good, but because the dataset is highly imbalanced, accuracy alone is not a reliable measure.

Important Finding

> **Logistic Regression was able to detect 73% of actual fraud transactions, but its very low fraud precision (5%) resulted in many false fraud alerts. This demonstrates why class imbalance must be handled carefully in fraud detection.**

The same Logistic Regression model was then trained using **Random Undersampling and Random Oversampling** to check whether balancing the training data improves Fraud **Precision, Recall, and F1 Score**.

---

12. Model Evaluation Metrics

The following metrics were used.

Accuracy

Measures the percentage of total predictions that are correct.

Precision

Measures how many transactions predicted as Fraud were actually Fraud.

Recall

Measures how many actual Fraud transactions were successfully detected.

F1 Score

Provides a balance between Precision and Recall.

Important for this project

For fraud detection, **Recall and F1 Score are more meaningful than Accuracy alone** because missing a fraudulent transaction can be costly.

---

13. Algorithm Comparison

After Logistic Regression, three additional algorithms were tested:

1. Decision Tree

A Decision Tree makes predictions using a series of conditions.

Advantages:

- Easy to understand
- Easy to interpret
- Does not require scaling

Limitation:

- Can overfit.

2. Random Forest

Random Forest combines multiple Decision Trees to produce a stronger prediction.

Advantages:

- Good performance
- Handles complex patterns
- More robust than one Decision Tree

3. KNN

K-Nearest Neighbors predicts a transaction based on similar transactions.

Advantages:

- Simple
- Easy to understand

Limitation:

- Requires feature scaling

- Can be slower with large datasets

---

14. Model Performance

After Logistic Regression, I tested **Decision Tree, Random Forest, and KNN** to find which algorithm performs better for credit card fraud detection.

1. Model Performance

Algorithm	Accuracy	Precision	Recall	F1 Score
Decision Tree	99.58%	45.83%	**48.72%**	47.23%
Random Forest	**99.77%**	**88.31%**	47.55%	**61.82%**
KNN	99.71%	68.77%	45.69%	54.90%

---

2. Decision Tree – Result

Accuracy: 99.58%
Precision: 45.83%
Recall: 48.72%
F1 Score: 47.23%

> "Decision Tree achieved 99.58% accuracy and the highest recall of 48.72% among these three models. However, its precision was only 45.83%, meaning it generated more false fraud predictions. Its F1 Score was also lower at 47.23%."

---

3. Random Forest – Result 🏆

Accuracy: 99.77%
Precision: 88.31%
Recall: 47.55%
F1 Score: 61.82%

> "Random Forest achieved the highest accuracy of 99.77%, the highest precision of 88.31%, and the highest F1 Score of 61.82%. Its recall was 47.55%, which was slightly lower than Decision Tree. However, it provided a much better balance between precision and recall."

---

4. KNN – Result

Accuracy: 99.71%
Precision: 68.77%
Recall: 45.69%
F1 Score: 54.90%

> "KNN achieved 99.71% accuracy and 68.77% precision. However, its recall was 45.69% and F1 Score was 54.90%, which were lower than Random Forest."

---

15. Final Comparison Table

Random Forest is the **best-performing model among these three algorithms**.

Rank	Algorithm	Accuracy	Precision	Recall	F1 Score
🥇	**Random Forest**	**99.77%**	**88.31%**	47.55%	**61.82%**
🥈	**KNN**	99.71%	68.77%	45.69%	54.90%
🥉	**Decision Tree**	99.58%	45.83%	**48.72%**	47.23%

Key Finding

Random Forest achieved the **highest Accuracy, Precision, and F1 Score**.

Decision Tree achieved slightly higher Recall, but its Precision and F1 Score were considerably lower.

Why Random Forest?

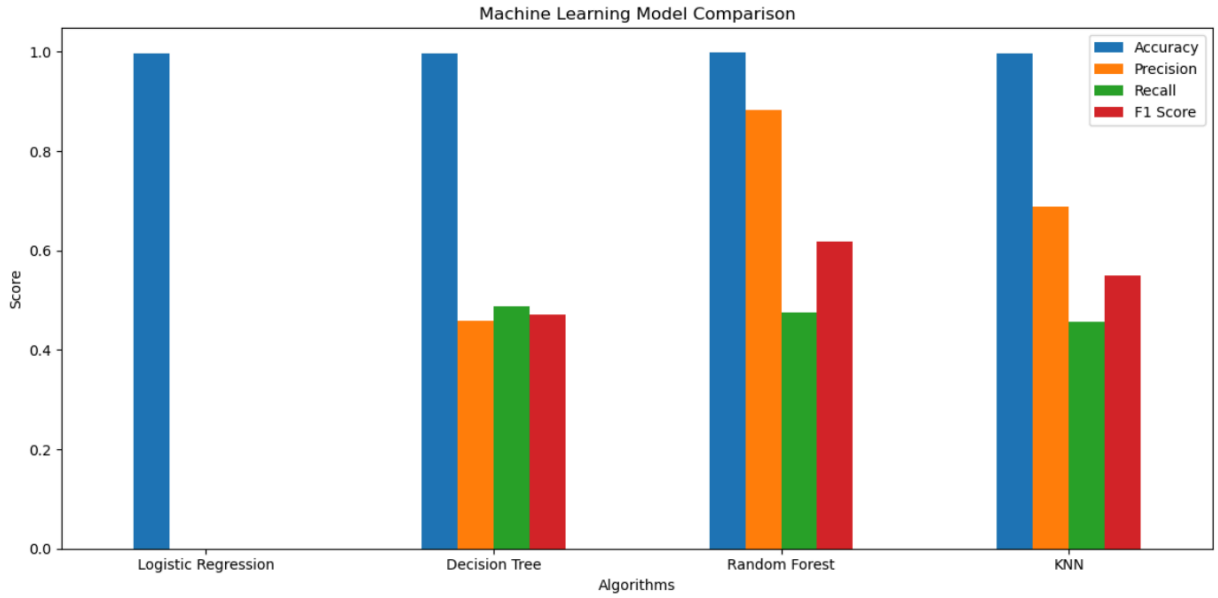
I selected Random Forest because it achieved the highest accuracy, precision, and F1 Score. Although Decision Tree had slightly higher recall, Random Forest provided a much better overall balance between precision and recall. Therefore, Random Forest was selected as the best model among the three.

---

16. Comparison Graph

The graph compares **Decision Tree, Random Forest, and KNN** based on Accuracy, Precision, Recall, and F1 Score.

Random Forest performs best overall, with the highest Accuracy (**99.77%**), Precision (**88.31%**), and F1 Score (**61.82%**). Decision Tree has slightly higher Recall (**48.72%**), while KNN performs between the two.



---

17. Confusion Matrix Comparison

Confusion Matrix Structure

	Predicted Non-Fraud	Predicted Fraud

Actual Non-Fraud	TN	FP
Actual Fraud	FN	TP

Where:

- **TN** = Correctly predicted Non-Fraud
- **TP** = Correctly predicted Fraud
- **FP** = Non-Fraud incorrectly predicted as Fraud
- **FN** = Fraud incorrectly predicted as Non-Fraud

Important Point

For fraud detection, **False Negatives are particularly important** because they represent actual fraudulent transactions that the model failed to identify.

---

18. Limitations

1. Class Imbalance

Fraud transactions are much fewer than Non-Fraud transactions, which makes fraud detection challenging.

2. Recall Can Be Improved

The selected Random Forest model has a Recall of **47.55%**, meaning some actual fraud transactions are still missed.

3. Limited Feature Selection

The initial model comparison used a selected set of features.

4. Hyperparameter Tuning

Extensive hyperparameter optimization was not performed.

5. Historical Data

The model is trained on historical transactions and may not automatically adapt to new fraud patterns.

6. No Real-Time Deployment

The current project focuses on model development and evaluation rather than real-time transaction monitoring.

---

19. Future Scope

The project can be improved by:

- Applying **SMOTE** for class imbalance.
- Performing **GridSearchCV/RandomizedSearchCV**.
- Testing **XGBoost and LightGBM**.
- Performing advanced feature engineering.
- Optimizing the classification threshold.
- Using **ROC-AUC and PR-AUC**.
- Building a real-time fraud detection system.
- Deploying the model using Flask or FastAPI.
- Creating a Power BI fraud monitoring dashboard.
- Periodically retraining the model using new transaction data.

---

20. Overall Conclusion ⭐

The Credit Card Fraud Detection project successfully demonstrated the use of Machine Learning for identifying fraudulent transactions. The project followed a complete Machine Learning workflow, including data cleaning, Exploratory Data Analysis, feature engineering, class imbalance analysis, balancing techniques, model training, and performance evaluation.

The dataset was found to be highly imbalanced, with Non-Fraud transactions significantly exceeding Fraud transactions. Random Undersampling and Random Oversampling were therefore explored to address the imbalance problem. Logistic Regression was initially tested to establish a baseline and was then retrained after balancing the training data.

In addition, Decision Tree, Random Forest, and KNN algorithms were implemented and compared using Accuracy, Precision, Recall, and F1 Score. The results showed that **Random Forest achieved the best overall performance**, with **99.77% Accuracy, 88.31% Precision, and 61.82% F1 Score**.

Although Decision Tree achieved a slightly higher Recall of 48.72%, Random Forest provided a stronger balance between Precision and Recall. Therefore, Random Forest was selected as the best model among the evaluated algorithms.

However, the Recall of 47.55% indicates that the model still misses a significant number of fraudulent transactions. This highlights the importance of further improvements through advanced class-balancing methods, feature engineering, hyperparameter tuning, threshold optimization, and advanced Machine Learning algorithms.

Overall, this project provided practical experience in **Python, Pandas, data preprocessing, EDA, feature engineering, imbalanced data handling, Machine Learning, model evaluation, and model selection**. It demonstrates how Machine Learning can support financial fraud detection and provides a strong foundation for developing a more accurate and real-time fraud detection system.